Name: Ishan Manekar class : A_A3_45

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import (classification_report, confusion_matrix, accuracy_score, precision_score, recall_score, f1
```

```
df = pd.read_csv("ai4i2020.csv")
df.head()
```

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0

```
df.tail()
```

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
9995	9996	M24855	M	298.8	308.4	1604	29.5	14	0	0	0	0	0	0
9996	9997	H39410	H	298.9	308.4	1632	31.8	17	0	0	0	0	0	0
9997	9998	M24857	M	299.0	308.6	1645	33.4	22	0	0	0	0	0	0
9998	9999	H39412	H	299.0	308.7	1408	48.5	25	0	0	0	0	0	0

Exploratory Data Analysis (EDA)

We will analyze the dataset's basic structure, descriptive statistics, missing values, target distributions, and correlation patterns.

```
# 1. General information and summary of data types and missing values
print("=== Dataset Info ===")
df.info()

print("\n=== Missing Values ===")
print(df.isnull().sum())

print("\n=== Descriptive Statistics ===")
display(df.describe())
```

```
=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   UDI                     10000 non-null  int64
 1   Product ID              10000 non-null  object
 2   Type                    10000 non-null  object
 3   Air temperature [K]     10000 non-null  float64
 4   Process temperature [K] 10000 non-null  float64
 5   Rotational speed [rpm]  10000 non-null  int64
 6   Torque [Nm]             10000 non-null  float64
 7   Tool wear [min]         10000 non-null  int64
 8   Machine failure         10000 non-null  int64
 9   TWF                     10000 non-null  int64
 10  HDF                     10000 non-null  int64
 11  PWF                     10000 non-null  int64
 12  OSF                     10000 non-null  int64
```

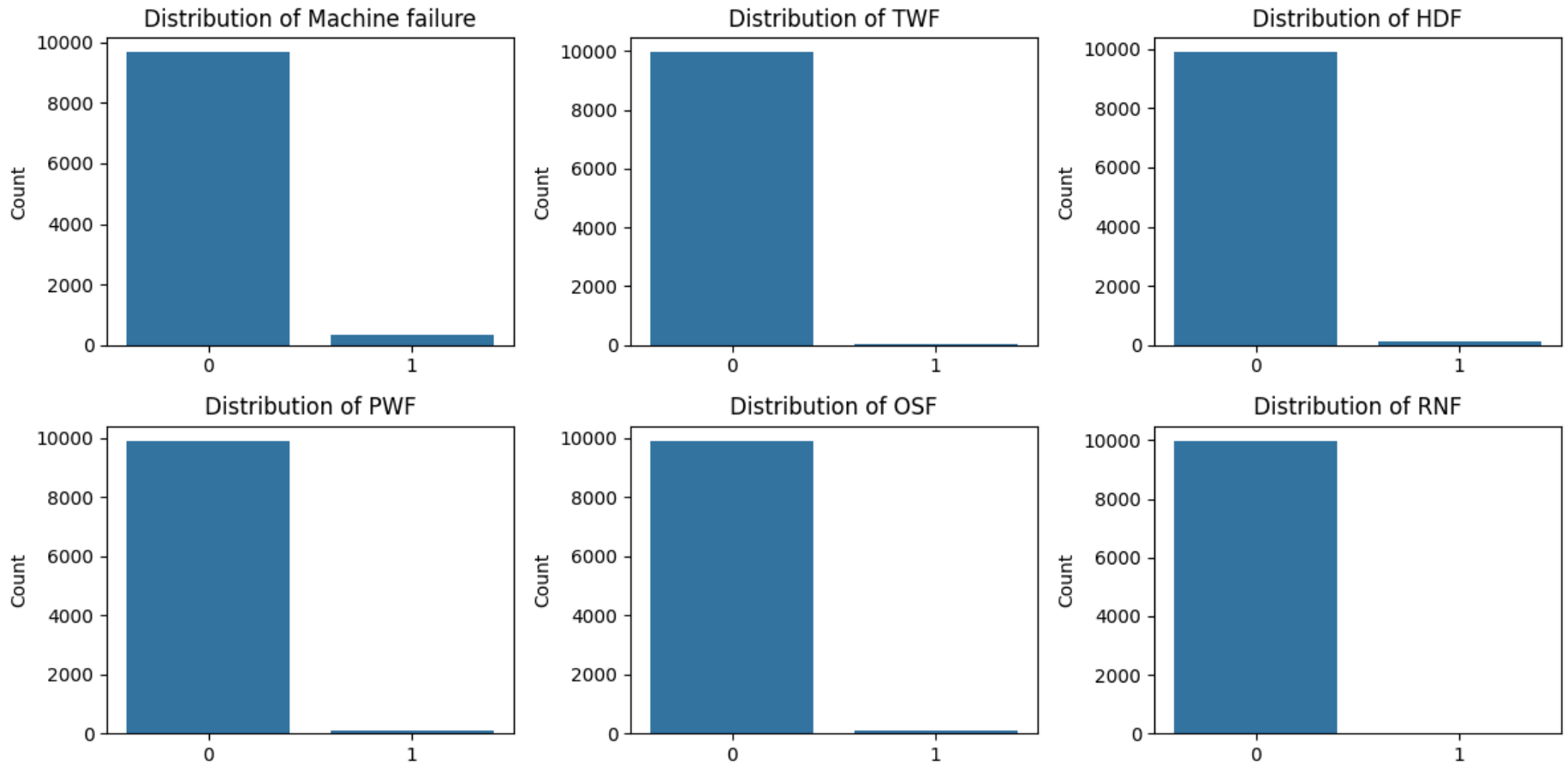
```
# 2. Distribution of the target 'Machine failure' and other failure modes
failure_cols = ['Machine failure', 'TWF', 'HDF', 'PWF', 'OSF', 'RNF']

plt.figure(figsize=(12, 6))
for i, col in enumerate(failure_cols, 1):
    plt.subplot(2, 3, i)
    sns.countplot(data=df, x=col)
    plt.title(f'Distribution of {col}')
    plt.xlabel('')
    plt.ylabel('Count')

plt.tight_layout()
plt.show()
```

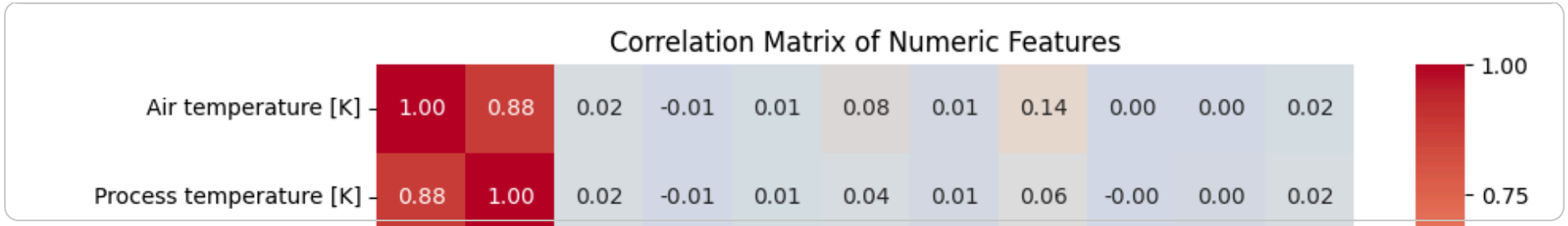
```
HDF                     0
PWF                     0
OSF                     0
RNF                     0
dtype: int64

=== Descriptive Statistics ===
```




```
# 3. Correlation heatmap of numeric features
numeric_cols = df.select_dtypes(include=[np.number]).columns
# Drop UDI as it's just an index identifier
numeric_cols = [col for col in numeric_cols if col != 'UDI']
```

```
plt.figure(figsize=(10, 8))
sns.heatmap(df[numeric_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm", square=True)
plt.title("Correlation Matrix of Numeric Features")
plt.show()
```



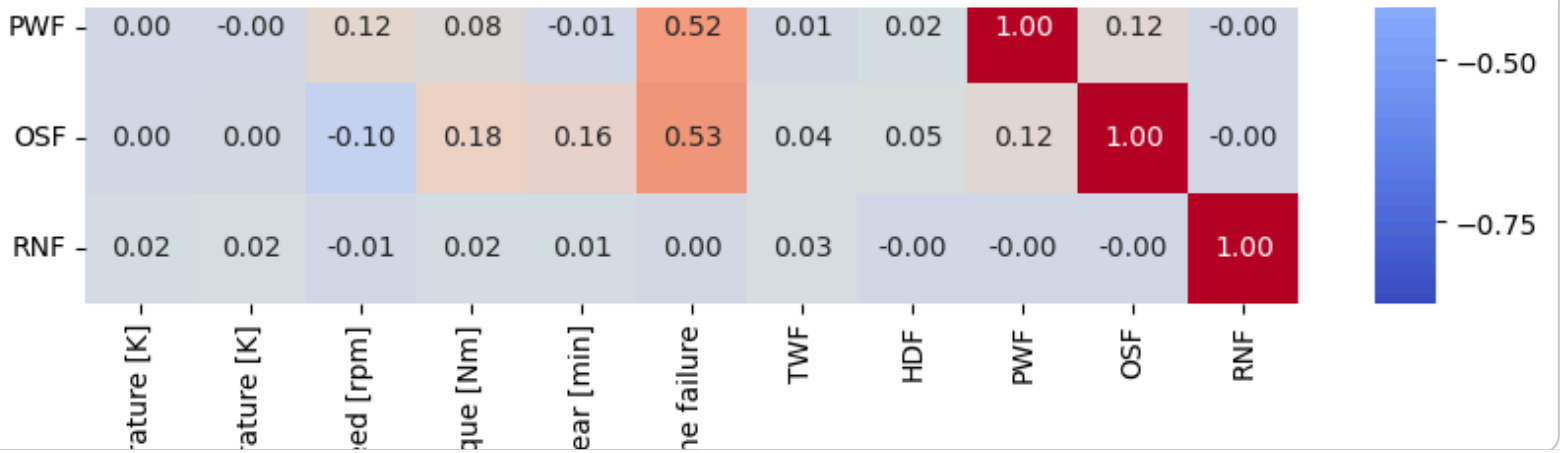
5. Additional EDA Visualizations

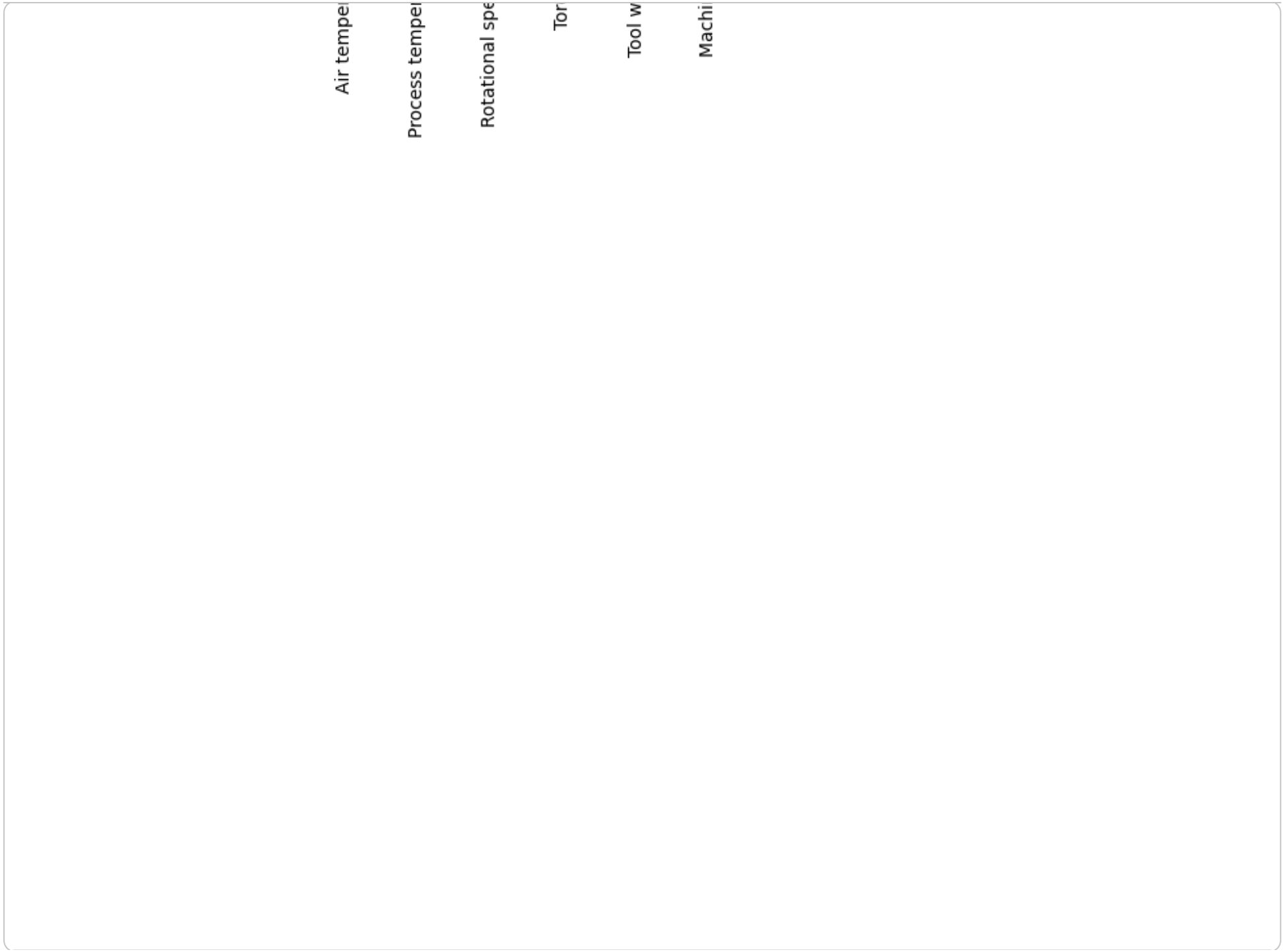
To better understand the feature relationships, we will plot the distributions of continuous features and how they relate to `Machine failure`.

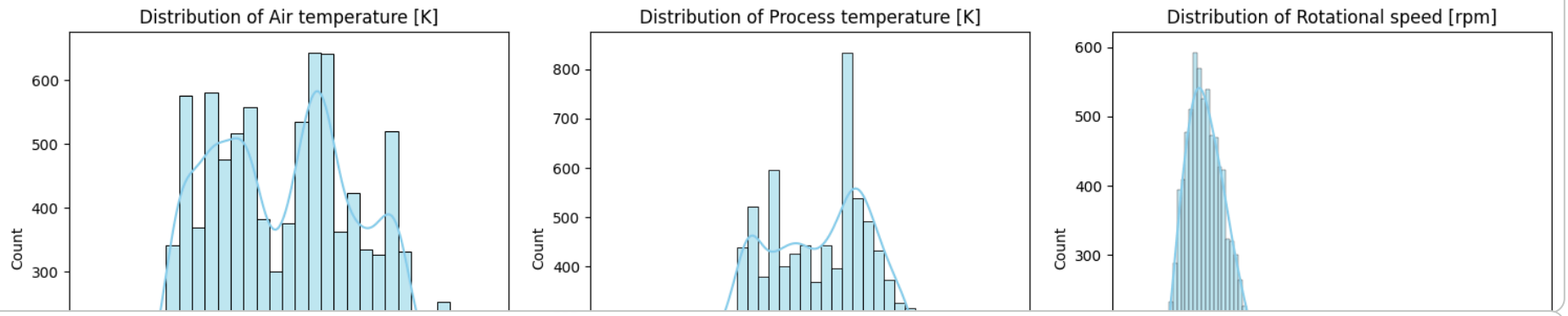
```
# Plotting distributions of continuous features
features_to_plot = ['Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]', 'Tool we

plt.figure(figsize=(15, 10))
for i, col in enumerate(features_to_plot, 1):
    plt.subplot(2, 3, i)
    sns.histplot(df[col], kde=True, color='skyblue')
    plt.title(f'Distribution of {col}')

plt.tight_layout()
plt.show()
```

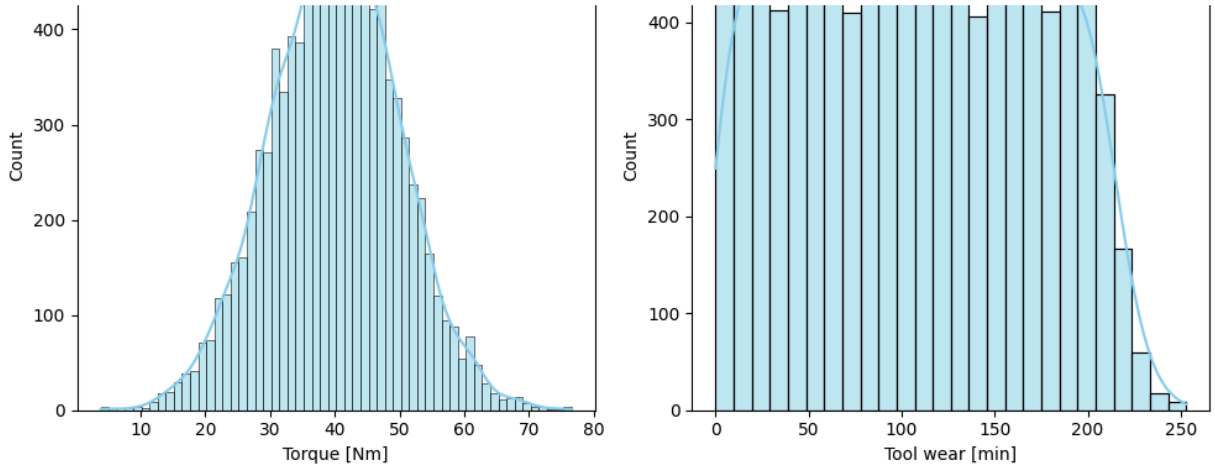


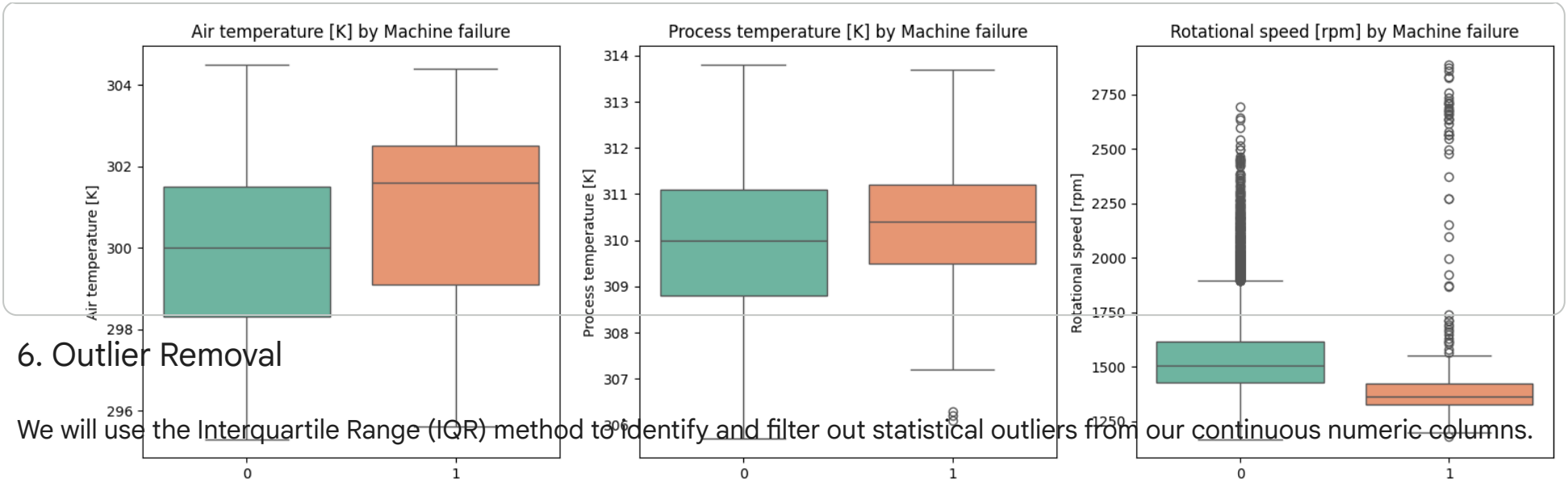

Air tempe
Process tempe
Rotational spe
Tor
Tool w
Machi




```
# Box plots grouped by Machine Failure to visualize differences
plt.figure(figsize=(15, 10))
for i, col in enumerate(features_to_plot, 1):
    plt.subplot(2, 3, i)
    sns.boxplot(data=df, x='Machine failure', y=col, palette='Set2')
    plt.title(f'{col} by Machine failure')

plt.tight_layout()
plt.show()
```





6. Outlier Removal

We will use the Interquartile Range (IQR) method to identify and filter out statistical outliers from our continuous numeric columns.

```
# Define numerical columns to filter outliers from
num_cols_for_outliers = ['Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]', 'To

# Keep a record of original size
original_shape = df.shape
print(f"Original dataset shape: {original_shape}")

# Implement IQR-based outlier removal
Q1 = df[num_cols_for_outliers].quantile(0.25)
Q3 = df[num_cols_for_outliers].quantile(0.75)
IQR = Q3 - Q1

# Define logical mask to keep only rows within 1.5 * IQR
mask = ~((df[num_cols_for_outliers] < (Q1 - 1.5 * IQR)) | (df[num_cols_for_outliers] > (Q3 + 1.5 * IQR))).any(axis=1)
df_cleaned = df[mask]

cleaned_shape = df_cleaned.shape
print(f"Cleaned dataset shape (without outliers): {cleaned_shape}")
print(f"Removed {original_shape[0] - cleaned_shape[0]} rows.")
```

```
Original dataset shape: (10000, 14)
Cleaned dataset shape (without outliers): (9541, 14)
Removed 459 rows.
```

8. Model Training on Cleaned Dataset

Now, we will train our SVM model on the dataset with outliers removed, perform hyperparameter tuning using GridSearchCV, and print the evaluation results.

```
# Define numeric and categorical features, and the target column
numeric_features = ['Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]', 'Tool we
categorical_features = ['Type']

features = numeric_features + categorical_features
target = 'Machine failure'

# Define features and target using the outlier-removed dataset
X_clean = df_cleaned[features]
y_clean = df_cleaned[target]

# Split into training and testing sets with stratification
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_clean, y_clean, test_size=0.2, random_state=42, stratify=y_clean
)

# Define the preprocessing pipeline
preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
])

# Reuse our preprocessing and pipeline configuration on the clean data
pipeline_clean = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', SVC(class_weight='balanced', random_state=42))
```

```
])

# Set up Grid Search parameters
param_grid = {
    'classifier__C': [0.1, 1, 10],
    'classifier__gamma': ['scale', 'auto']
}

grid_search_clean = GridSearchCV(pipeline_clean, param_grid, cv=5, scoring='f1', n_jobs=-1)
grid_search_clean.fit(X_train_c, y_train_c)

print("Best parameters found:", grid_search_clean.best_params_)
print("Best cross-validation F1-score on cleaned data:", grid_search_clean.best_score_)
```

```
Best parameters found: {'classifier__C': 10, 'classifier__gamma': 'scale'}
Best cross-validation F1-score on cleaned data: 0.4678448408930837
```

```
# Evaluate the model trained on cleaned data
best_model_clean = grid_search_clean.best_estimator_
y_pred_clean = best_model_clean.predict(X_test_c)

# Calculate individual metric scores
acc = accuracy_score(y_test_c, y_pred_clean)
prec = precision_score(y_test_c, y_pred_clean)
rec = recall_score(y_test_c, y_pred_clean)
f1 = f1_score(y_test_c, y_pred_clean)

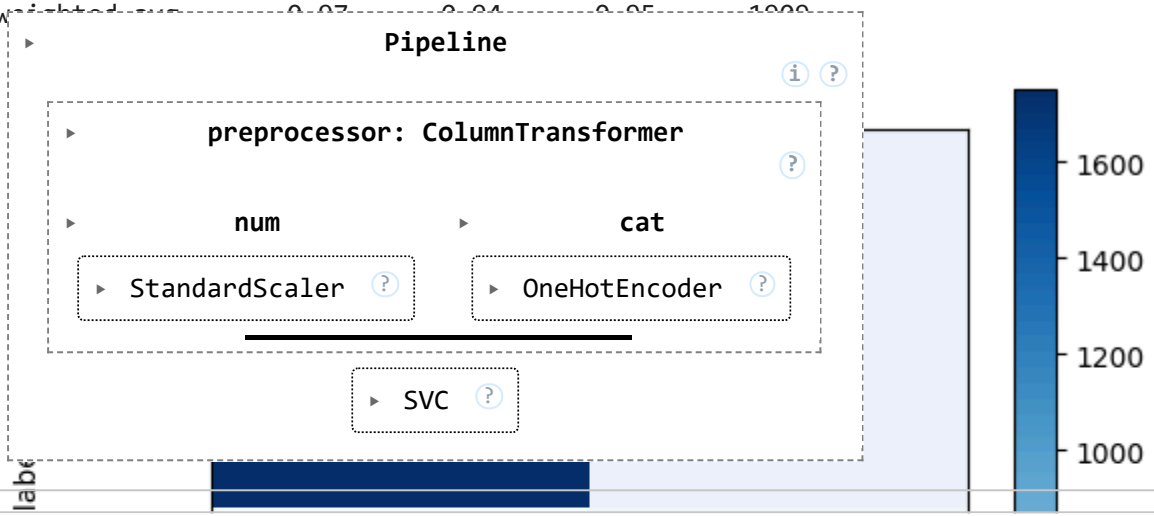
print("=== Performance Metrics (Cleaned Data) ===")
print(f"Accuracy:  {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall:    {rec:.4f}")
print(f"F1-Score:  {f1:.4f}")
print("\n=== Classification Report ===")
print(classification_report(y_test_c, y_pred_clean))

# Plot confusion matrix using ConfusionMatrixDisplay
fig, ax = plt.subplots(figsize=(6, 5))
```

```
cm_clean = confusion_matrix(y_test_c, y_pred_clean)
disp_clean = ConfusionMatrixDisplay(confusion_matrix=cm_clean, display_labels=['No Failure', 'Failure'])
disp_clean.plot(cmap='Blues', ax=ax, values_format='d')
plt.title('Confusion Matrix (Cleaned Data)')
plt.tight_layout()
plt.show()
```

```
=== Performance Metrics (Cleaned Data) ===
Accuracy:  0.9387
```

```
# Kernel Linear
linear_svm = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('classifier', SVC(kernel='linear', class_weight='balanced', random_state=42))
    ]
)
linear_svm.fit(X_train_c, y_train_c)
```




```
y_pred_linear = linear_svm.predict(X_test_c)
```

```
accuracy_linear = accuracy_score(y_test_c, y_pred_linear)
precision_linear = precision_score(y_test_c, y_pred_linear,zero_division = 0)
recall_linear = recall_score(y_test_c, y_pred_linear,zero_division = 0)
f1_linear = f1_score(y_test_c, y_pred_linear,zero_division = 0)
print("Accuracy:", accuracy_linear)
print("Precision:", precision_linear)
```

```
print("Recall:", recall_linear)
print("F1 Score:", f1_linear)
```

```
Accuracy: 0.8161341016238869
Precision: 0.11597938144329897
Recall: 0.8490566037735849
F1 Score: 0.20408163265306123
```

```
# 1. Polynomial Kernel SVM
poly_svm = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('classifier', SVC(kernel='poly', degree=3, class_weight='balanced', random_state=42))
    ]
)
poly_svm.fit(X_train_c, y_train_c)
y_pred_poly = poly_svm.predict(X_test_c)

accuracy_poly = accuracy_score(y_test_c, y_pred_poly)
precision_poly = precision_score(y_test_c, y_pred_poly, zero_division=0)
recall_poly = recall_score(y_test_c, y_pred_poly, zero_division=0)
f1_poly = f1_score(y_test_c, y_pred_poly, zero_division=0)

print("=== Polynomial SVM (Degree 3) ===")
print("Accuracy: ", accuracy_poly)
print("Precision:", precision_poly)
print("Recall:   ", recall_poly)
print("F1 Score: ", f1_poly)
```

```
=== Polynomial SVM (Degree 3) ===
Accuracy:  0.9214248297537978
Precision: 0.24338624338624337
Recall:    0.8679245283018868
F1 Score:  0.38016528925619836
```

```
# 2. Sigmoid Kernel SVM
sig_svm = Pipeline(
```

```
    steps=[
        ('preprocessor', preprocessor),
        ('classifier', SVC(kernel='sigmoid', class_weight='balanced', random_state=42))
    ]
)
sig_svm.fit(X_train_c, y_train_c)
y_pred_sig = sig_svm.predict(X_test_c)

accuracy_sig = accuracy_score(y_test_c, y_pred_sig)
precision_sig = precision_score(y_test_c, y_pred_sig, zero_division=0)
recall_sig = recall_score(y_test_c, y_pred_sig, zero_division=0)
f1_sig = f1_score(y_test_c, y_pred_sig, zero_division=0)

print("=== Sigmoid SVM ===")
print("Accuracy: ", accuracy_sig)
print("Precision:", precision_sig)
print("Recall:   ", recall_sig)
print("F1 Score: ", f1_sig)
```

```
=== Sigmoid SVM ===
Accuracy:  0.6589837611314825
Precision: 0.05505952380952381
Recall:    0.6981132075471698
F1 Score:  0.10206896551724139
```

```
# Define the RBF SVM using the best estimator from our previous grid search
rbf_svm = grid_search_clean.best_estimator_
y_pred_rbf = rbf_svm.predict(X_test_c)

# Calculate metrics for RBF SVM
accuracy_rbf = accuracy_score(y_test_c, y_pred_rbf)
precision_rbf = precision_score(y_test_c, y_pred_rbf, zero_division=0)
recall_rbf = recall_score(y_test_c, y_pred_rbf, zero_division=0)
f1_rbf = f1_score(y_test_c, y_pred_rbf, zero_division=0)

print("=== RBF SVM (Tuned) ===")
print("Accuracy: ", accuracy_rbf)
```

```
print("Precision:", precision_rbf)
print("Recall:   ", recall_rbf)
print("F1 Score: ", f1_rbf)
```

```
=== RBF SVM (Tuned) ===
Accuracy:  0.9387113672079623
Precision: 0.2866666666666667
Recall:    0.8113207547169812
F1 Score:  0.4236453201970443
```

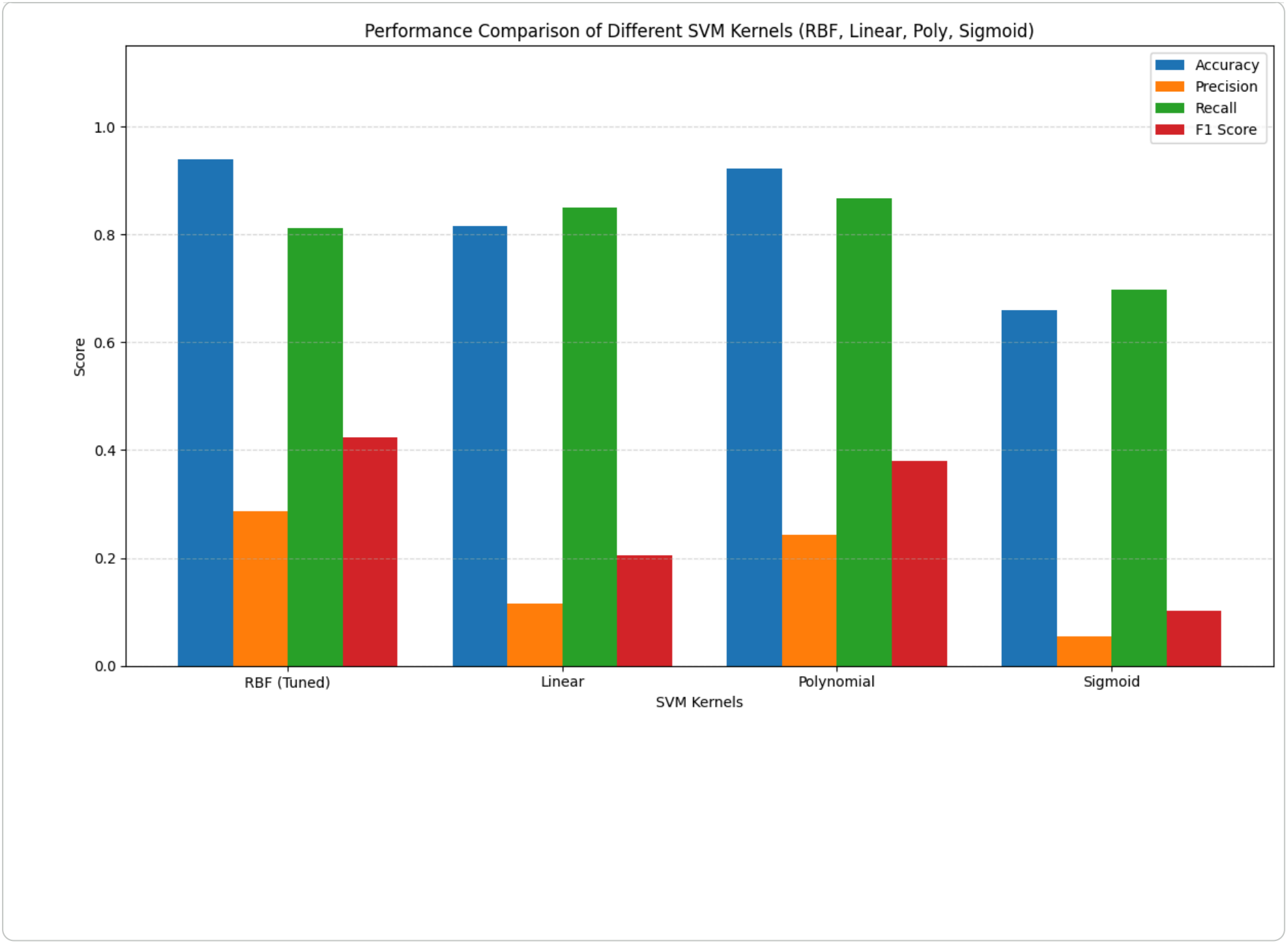
```
# Consolidated Performance Plot for all 4 SVM Kernels
kernels = ['RBF (Tuned)', 'Linear', 'Polynomial', 'Sigmoid']

accuracy_scores = [accuracy_rbf, accuracy_linear, accuracy_poly, accuracy_sig]
precision_scores = [precision_rbf, precision_linear, precision_poly, precision_sig]
recall_scores = [recall_rbf, recall_linear, recall_poly, recall_sig]
f1_scores = [f1_rbf, f1_linear, f1_poly, f1_sig]

x = np.arange(len(kernels))
width = 0.2

plt.figure(figsize=(12, 7))
plt.bar(x - 1.5*width, accuracy_scores, width, label='Accuracy', color='#1f77b4')
plt.bar(x - 0.5*width, precision_scores, width, label='Precision', color='#ff7f0e')
plt.bar(x + 0.5*width, recall_scores, width, label='Recall', color='#2ca02c')
plt.bar(x + 1.5*width, f1_scores, width, label='F1 Score', color='#d62728')

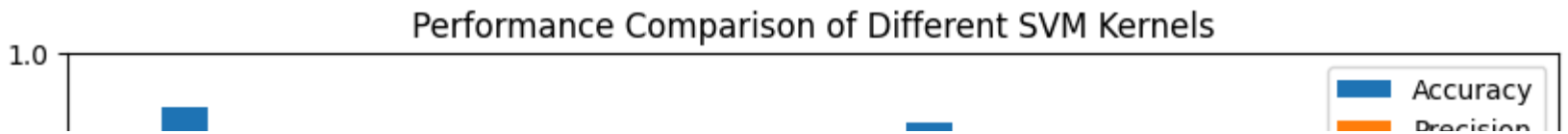
plt.xlabel('SVM Kernels')
plt.ylabel('Score')
plt.title('Performance Comparison of Different SVM Kernels (RBF, Linear, Poly, Sigmoid)')
plt.xticks(x, kernels)
plt.ylim(0, 1.15)
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```


Performance Comparison of Different SVM Kernels (RBF, Linear, Poly, Sigmoid)
Accuracy
Precision
Recall
F1 Score
Score
1.0
0.8
0.6
0.4
0.2
0.0
RBF (Tuned)
Linear
Polynomial
Sigmoid
SVM Kernels

```
comparison = pd.DataFrame({
    'Kernel': ['RBF (Tuned)', 'Linear', 'Polynomial', 'Sigmoid'],
    'Accuracy': [accuracy_rbf, accuracy_linear, accuracy_poly, accuracy_sig],
    "Precision": [precision_rbf, precision_linear, precision_poly, precision_sig],
    "Recall": [recall_rbf, recall_linear, recall_poly, recall_sig],
    "F1 Score": [f1_rbf, f1_linear, f1_poly, f1_sig]
})
comparison
```

	Kernel	Accuracy	Precision	Recall	F1 Score
0	RBF (Tuned)	0.938711	0.286667	0.811321	0.423645
1	Linear	0.816134	0.115979	0.849057	0.204082
2	Polynomial	0.921425	0.243386	0.867925	0.380165
3	Sigmoid	0.658984	0.055060	0.698113	0.102069

```
comparison_plot =comparison.set_index('Kernel')
comparison_plot.plot(kind="bar", figsize=(10,6))
plt.title("Performance Comparison of Different SVM Kernels")
plt.ylabel("Score")
plt.ylim(0,1)
plt.show()
```



Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.